Основы языка Kotlin для мобильной разработки



Павел Галанин
Android-разработчик

Содержание

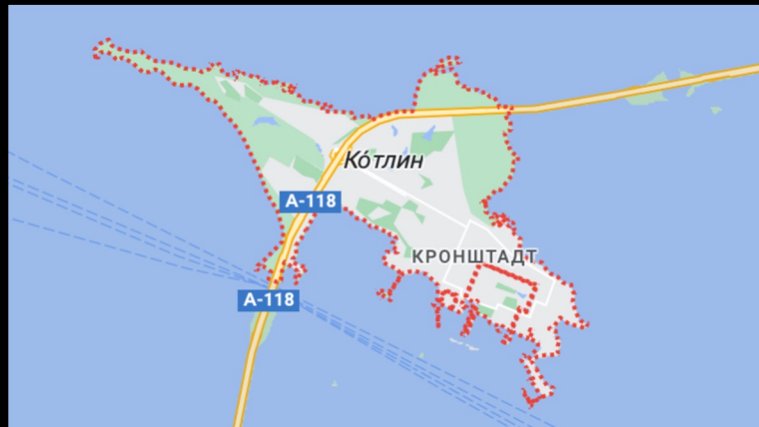
- Базовые конструкции языка
- Классы и объекты
- Generic и коллекции
- Разное в Kotlin

Зачем учить язык, если ИИ пишет код?

- Понимать результат: что делает код и какие у него ограничения.
- Точно ставить задачи: формулировать поведение и технические требования
- Проверять решения: замечать ошибки и выбирать подходящую реализацию
- Развивать приложение: осознанно менять код и разбираться в сбоях

О Kotlin

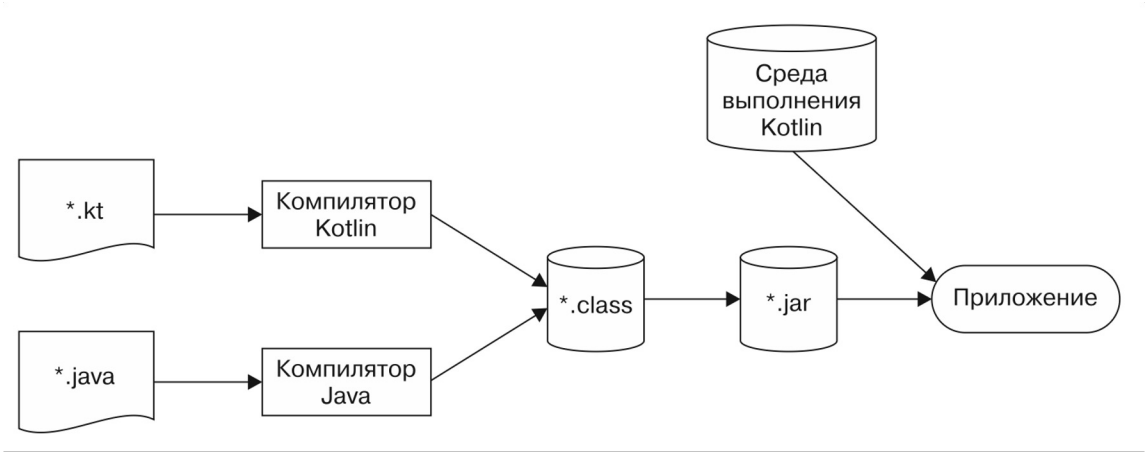
- Kotlin — это современный язык программирования, разработанный компанией JetBrains.
- Разрабатывается с 2010 года
- Первый релиз – 2016
- Основное применение – Android, Backend
- С 2019 года – Kotlin-first разработка под Android



Kotlin и Java

Kotlin и Java – языки, которые идут в паре, их **можно использовать в одном проекте**.

И Kotlin, и Java компилируются в набор `.class` файлов компилятором kotlin и java соответственно, из которых в дальнейшем собирается `.jar`-файл.



Базовые конструкции языка

Функции

Функция принимает параметры и в зависимости от них возвращает результат и/или выполняет какие либо действия.

```
fun max(a: Int, b: Int): Int {
    if (a > b) {
        return a
    } else {
        return b
    }
}
```

Объявление функции должно содержать:

- fun
- название функции
- набор параметров
- тип возвращаемого значения

Функции

if в Kotlin – выражение, оно **умеет возвращать результат**

Поэтому мы можем сократить нашу функцию

```
fun max(a: Int, b: Int): Int {
    return if (a > b) a else b
}
```

Функции

Если функция должна возвращать результат и укладывается в одну конструкцию, мы можем воспользоваться следующим синтаксисом:

```
fun max(a: Int, b: Int): Int = if (a > b) a else b
```

Функции

Или же опустить явное указание типа возвращаемого значения, потому что Kotlin может вычислить его сам:

```
fun max(a: Int, b: Int) = if (a > b) a else b
```

Функции – Unit

Если не указывать возвращаемое значение и не использовать синтаксис с =, функция вернет `object Unit`, то есть, фактически, ничего.

```
fun writeToConsole() {
    print("Hello, world!")
}
```

```
fun writeToConsole(): Unit {
    print("Hello, world!")
    return Unit
}
```

Функции – default params

В Kotlin у функций есть возможность задавать параметрам значение по умолчанию

```
fun defaultParams(
    a: String,
    b: String = "2",
    c: String = "3",
) { ... }


defaultParams("param a", "param b", "param c")
defaultParams(a = "param a", c = "param c", b = "param b")
```

Функции – default params

```
defaultParams(
    a = "param a",
    // пропустили b
    c = "param c",
)

defaultParams(
    "param a", // a - не именованый
    // пропустили b
    c = "param c",
)
```

Функции – default params

```
CircularProgressDrawable(true)
```

Функции – default params

```
CircularProgressDrawable(useCheckMark = true)
```

Функции – vararg

Vararg — это способ передать переменное количество аргументов в функцию.

Модификатор vararg перед параметром означает, что в функцию можно передать любое количество значений этого типа, они будут собраны в массив.

```
fun greetAll(vararg names: String) {
    for (name in names) {
        println("Hello, $name!")
    }
}

greetAll("Alice", "Bob", "Charlie")
```

Функции – vararg

- Можно использовать только один vararg параметр на функцию, и он должен быть последним
- Можно передать уже готовый массив с помощью * (спред-оператор):

```
val array = arrayOf("Anna", "Oleg")
greetAll(*array)
```

Внутри функции names — это обычный массив (Array<String>), и с ним можно работать как обычно.

Переменные и типы

Бывает два вида типов:

Примитивы – это базовые типы: целое число (Int/Long), символ (Char), логическое значение правда или ложь (Boolean).

Объекты же – "составные" типы, состоящие из примитивов и/или других объектов и обладающих поведением. Примером такого типа может служить `List` – список.

Главное отличие – примитивы передаются "по значению", а объекты – "по ссылке".

Переменные и типы

Отдельных примитивных типов, как в Java и в других языках, в Kotlin нету, в Kotlin мы со всеми типами работаем как с объектами:

```
/**
 * Represents a 32-bit signed integer.
 * On the JVM, non-nullable values of this type are represented
 * as values of the primitive type `int`.
 */
public class Int private constructor() : Number(), Comparable<Int> {
    ...
}
```

Переменные и типы

```
val boolTrue: Boolean = true
val number: Int = 1
val longNumber: Long = 1
val shortNumber: Short = 1
val byteNumber: Byte = 1
val doubleNumber: Double = 1.0
val floatNumber: Float = 1f
val charA = 'a'
```

Переменные

Переменные объявляются с помощью ключевого слова val или var.

val — это неизменяемая переменная (аналог final в Java). Значение присваивается один раз, далее доступна только для чтения.

var — это изменяемая переменная. Ее значение можно менять.

```
val a = 0
var b = 1

// a = 2 <- не сможем изменить значение
b = 3
```

Переменные

```
val a = 0
var b = 1

// a = 2 <- не сможем изменить значение
b = 3

var c: Int = 2

val d: Int
d = 3

var e: Int
e = 4
```

Переменные

В kotlin переменные хранят внутри себя какое-то значение: примитивное (число, char, boolean,...), или же ссылку на объект.

```
var a = 0
var b = a

a = 3

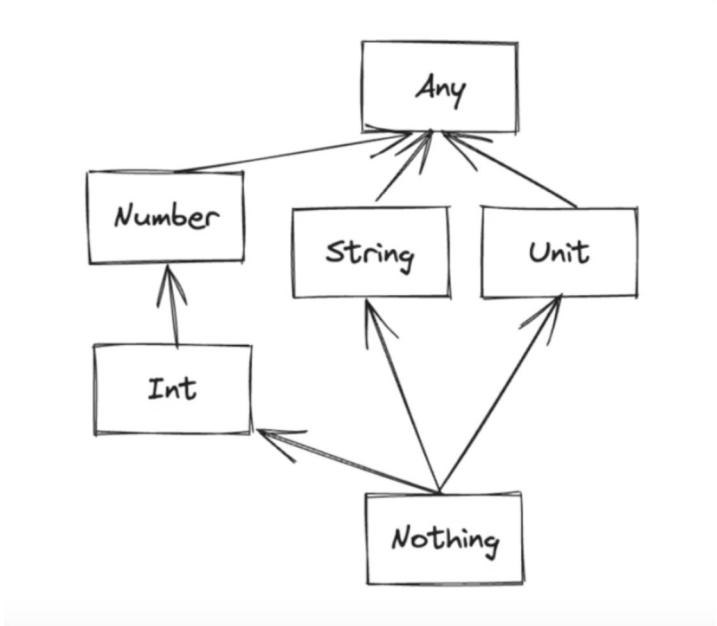
assert(b == 0)
```

```
// Создаем объект List<Int> и сохраняем
// ссылку на него в переменную list1
val list1 = mutableListOf(1, 2, 3)
// Создаем переменную list2 и сохраняем
// в нее ссылку на объект List<Int>,
// созданный ранее
val list2 = list1

list1.add(4)

assert(list2 == listOf(1, 2, 3, 4))
```

Встроенные типы



Встроенные типы – Nothing

```kotlin
/**
 * Always throws [NotImplementedError] stating that operation is not implemented.
 */

@kotlin.internal.InlineOnly
public inline fun TODO(): Nothing = throw NotImplementedError()
```

```kotlin
@SinceKotlin( version: "1.4")
override fun fail(message: String?, cause: Throwable?): Nothing {
    Assertions.fail<Any?>(message, cause)
    // should not get here
    throw AssertionError(message, cause)
}
```

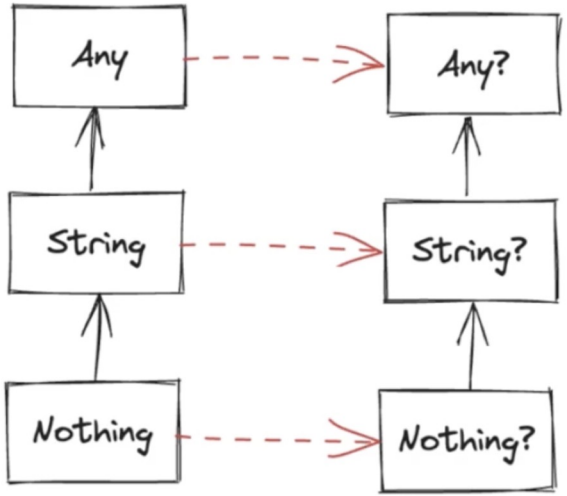
Nullable

В Kotlin мы можем явно указать, сможет ли переменная принимать значение null или нет.

Тип nullable, если после него стоит символ вопросительного знака:

```
val a: Int = 0 // `a` нельзя присвоить null
val b: Int? = null // а `b` - можно присвоить null
```

Nullable



Nullable – smartcast

```
fun calculateA(): Int? { ... }

fun doSmth(a: Int) { ... }

val a = calculateA()
if (a != null) {
    // a автоматически преобразован к Int
    doSmth(a)
} else {
    // ...
}
```

Nullable – fallback

```
fun calculateA(): Int? { ... }

fun doSmth(a: Int) { ... }

// Тип Int, тк если calculateA()
// вернет null, выражение вернет
// fallback-значение 0.
val a: Int = calculateA() ?: 0
doSmth(a)
```

```
// Тип Int, тк если calculateA()
// вернет null, будет проброшено
// исключение
val a: Int = calculateA()
    ?: throw IllegalStateException("")
doSmth(a)
```

```
// Тип Int, тк если calculateA()
// вернет null, функция прекратит
// свою работу
val a: Int = calculateA()
    ?: return
doSmth(a)
```

Nullable – если не null

```
data class Item(
    val title: String,
    val subtitle: String,
)

val items: List<Item>? = loadItems()

// Если бы мы использовали items.get,
// получили бы ошибку компиляции.
//
// Если бы мы использовали items.get
// в Java, получили бы
// NullPointerException в рантайме.
val first: Item? = items?.get(0)
```

Nullable – assert

```
val items: List<Int>? = loadItems()

items!!.get(0)
```

Nullable – примитивы

Примитивы Int, Char, и тд также могут быть Nullable, но в таком случае они будут представлены в виде объекта.

Условные операторы – if

У if есть удобная особенность - он умеет кастить объект к определенному типу после проверки.

```
fun doSmth(a: Any) {
    if (a is String) {
        // a - String в этом блоке кода
    } else if (a is Int) {
        // a - Int в этом блоке кода
    }
}
```

Условные операторы – when

when в Kotlin — это расширенная версия switch из Java и других языков, но более мощная и выразительная.

Можно использовать when ровно также, как if/else if/.../else, это будет более компактный и простой вариант:

```
when {
    condition1 -> { ... }
    condition2 -> { ... }
    condition3 -> { ... }
    else -> { ... }
}
```

Условные операторы – when

Можно работать с конкретной переменной

```
when (a) {
    1 -> {}
    2 -> {}
    else -> {}
}
```

```
fun doSmth(a: Any) {
    when (a) {
        is String -> {
            // a - String в этом блоке кода
        }
        is Int -> {
            // a - Int в этом блоке кода
        }
    }
}
```

Циклы – while

В kotlin есть стандартные циклы while, do while, for, и, фишка Kotlin – Ranges.

```
var condition: Boolean = ...

while (condition) {
    // do smth
}
```

```
do {
    ...
} while (condition)
```

Циклы – for

```
val list = listOf(0, 1, 2, 3, 4)

for (i in 0..(list.size - 1)) {
    println(list[i])
}
```

```
val list = listOf(0, 1, 2, 3, 4)

for (i in 0 until list.size) {
    println(list[i])
}
```

Но компилируется все так:

```
var i: Int = 0
while (i < list.size) {
    println(list[i])
    i += 1
}
```

Циклы – for

```
val list = listOf(1, 2, 3, 4, 5)
for (item in list) {
    println(item)
}
```

Лямбда-выражения

Лямбда – инстанса упрощенно описанного класса, в который мы забрали как поля все, что мы принимаем как параметр и который имеет единственную функцию `invoke()`.

```
button.setOnClickListener({ view ->
    // do smth on click
})
```

Лямбда-выражения

1. Отложенное исполнение операций

```
button.setOnClickListener({ view ->
    // do smth on click
})

// То же самое
button.setOnClickListener { view ->
    // do smth on click
}
```

2. Переиспользование операций

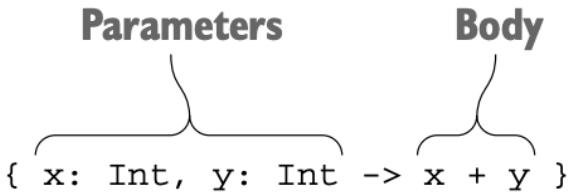
```
val largestNumber = listOf(10, 40, 5).maxBy { it }
val lastWord = listOf("Computer", "Code").maxBy { it }


val oldest = people.maxBy { human: Human ->
    human.age
}

val oldest = people.maxBy { human ->
    human.age
}

val oldest = people.maxBy { it.age }
```

Синтаксис лямбда-выражений




```
val oldest = people.maxBy { human: Human ->
    human.age
}
```

```
val oldest = people.maxBy { human ->
    human.age
}
```

```
val oldest = people.maxBy { it.age }
```

Лямбда-выражения

```
button.setOnClickListener { view ->
    // do smth on click
}
```

```
button.setOnClickListener(
    object : Function1<View, Unit> {
        override operator fun invoke(view: View) {
            // do something
        }
    }
)
```

Классы и объекты

Классы и объекты

- Класс определяет «формат» для однотипных сущностей
- В общем случае класс имеет какие-то данные (поля) и определяет какое-то поведение (методы)
- Объекты – конкретные «представители» классов, «инстансы»

```
class Human {
    private val name: String
    private val weight: Int

    constructor(name: String, weight: Int) {
        this.name = name
        this.weight = weight
    }

    fun sayYourName() {
        println("My name is $name")
    }
}

val pavel = Human( name: "Pavel",  weight: 80)
pavel.sayYourName() // "My name is Pavel"

```

Класс, объект

Простой пример класса – Human, он обладает свойствами (name, weight) и поведением (sayName()).

```
class Human {
    private val name: String
    private val weight: Int

    constructor(name: String, weight: Int) {
        this.name = name // this - обращаемся к инстансу класса
        this.weight = weight
    }

    fun sayName() {
        println("My name is $name")
    }
}

val human = Human(name = "Name", weight = 80)
human.sayName()
```

Конструктор класса

Два вида конструкторов:

- **Первичный** (primary) конструктор – если есть, то обязательно должен быть вызван
- **Вторичный** (secondary) конструктор – "альтернативный" способ создания инстанса класса, может быть 0+ вторичных конструкторов.

Конструктор класса

```
class User1 {
    val _name: String

    constructor(name: String) { // <- первичный конструктор
        this.name = name // this - обращение к текущему инстансу класса
    }
    // Это валидное решение, тем не менее, kotlin будет подсвечивать,
    // что первичный конструктор все таки должен находится рядом
    // с объявлением функции.
}

// kotlin позволяет объявить параметры первичного конструктора прямо в
// месте объявления класса.
class User2 constructor(name: String) {
    val name: String = name
}
```

Конструктор класса

```
// Более того, в таком случае нам даже не обязательно явно прописывать
// ключевое слово constructor, это все еще первичный конструктор
class User3(name: String) {
    val name: String = name
}

class User4(name: String) { // <- первичный конструктор
    val name: String = name

    // вторичный конструктор, может вызывать первичный или любой другой
    // конструктор, если бы они тут были.
    // Вторичный конструктор при желании может вызвать любой другой.
    constructor(nameSurname: Pair<String, String>) : this(nameSurname.first())
}

class User5(val name: String) // Идеальная ситуация
```

Блок инициализации

В классе может присутствовать блок инициализации, который вызовется после вызова конструктора.

```
class Shape(rawColor: String) {
    val color: String

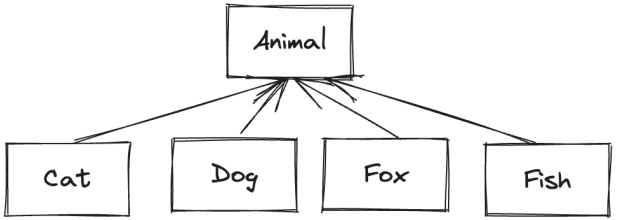
    init {
        println("Create color: $color")
        color = rawColor
    }
}
```

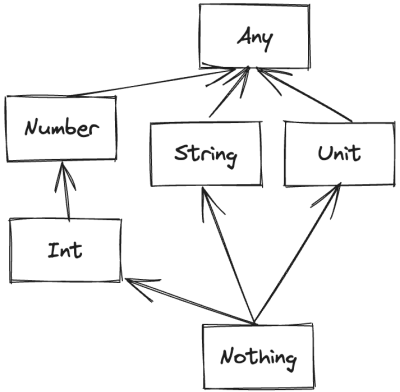
Поля класса

Поля в общем случае хранят состояние объекта.

```
class User(val name: String) {
    var address: String = "unspecified"
        get(): String {
            println("Address was requested")
            return field
        }
        set(value: String) {
            println("Address was changed for name: $name : $field -> $value")
            field = value
        }
}
```

Наследование






```kotlin
abstract class Animal(val weight: String) {
    abstract fun whatDoYouSay()
}

class Cat(weight: String) : Animal(weight) {
    override fun whatDoYouSay() {
        println("Meow, meow")
    }
}

class Dog(weight: String) : Animal(weight) {
    override fun whatDoYouSay() {
        println("Woof, woof")
    }
}
class Fox(weight: String) : Animal(weight) {
    override fun whatDoYouSay() {
        println("https://www.youtube.com/watch?v=jofNR_WkoCE")
    }
}

class Fish(weight: String) : Animal(weight) {
    override fun whatDoYouSay() {
        throw NotImplementedError()
    }
}
```

Абстрактные классы

Абстрактный класс — это класс, который не может быть создан напрямую и может содержать незаконченные (abstract) методы, которые обязаны быть реализованы в подклассах. Используется для задания общей логики и структуры для наследников.

```
abstract class Animal(val weight: String) {
    abstract fun whatDoYouSay()
}

class Cat(weight: String) : Animal(weight) {
    override fun whatDoYouSay() {
        println("Meow, meow")
    }
}

class Dog(weight: String) : Animal(weight) {
    override fun whatDoYouSay() {
        println("Woof, woof")
    }
}
```

Абстрактные классы – полиморфизм

Зачем это нужно?

У нас есть обязательство, что данный тип имеет определенное поведение, при этом реализация поведения зависит от конкретной сущности.

Два хороших, но разных примера – List<T> и паттерн стратегии.

```
fun main() {
    val list1: ArrayList<Int> = ...
    val list2: LinkedList<Int> = ...

    printList(list1)
    printList(list2)
}

fun printList(list: List<Int>) {
    for (item in list) {
        println(item)
    }
}
```

Абстрактные классы – полиморфизм

Эта особенность, возможность использования базового типа List называется **полиморфизм**.

Полиморфизм – способность программы идентично использовать объекты с одинаковым интерфейсом без информации о конкретном типе этого объекта.

```
fun main() {
    val list1: ArrayList<Int> = ...
    val list2: LinkedList<Int> = ...

    printList(list1)
    printList(list2)
}

fun printList(list: List<Int>) {
    for (item in list) {
        println(item)
    }
}
```

Интерфейс

Интерфейс — это контракт, который определяет набор методов и свойств, без хранения состояния. Классы могут реализовывать несколько интерфейсов, а интерфейсы могут содержать реализацию по умолчанию.

```
interface Clearable {
    fun clear() {}
}

class SomeTaskExecutorImpl : TaskExecutor(), Clearable {
    val currentRequests: List<Request>

    override fun execute() {
        val request = ...
        currentRequests.add(request)
    }

    override fun clear() {
        currentRequests.forEach {
            it.cancel()
        }
    }
}
```

Абстрактные классы/интерфейсы – полиморфизм

Еще один пример – стратегия.

Возьмем абстрактный RatingBar.
От фичи к фиче могут быть разные сценарии использования компонента, поэтому у нас объявлен RatingBarFillStrategy с методами calculateFilledAreaPx(params), а конкретная фича просто передает нужную реализацию этой стратегии в дизайн-компонент.

```
interface FillStrategy {
    fun calculateFilledAreaWidthPx(
        rating: Float,
        ...
    ): Int

    class AsIs : FillStrategy {
        override fun calculateFilledAreaWidthPx(
            rating: Float,
            ...
        ): Int = ...
    }

    class IntegerStars : FillStrategy {
        override fun calculateFilledAreaWidthPx(
            rating: Float,
            ...
        ): Int = filledStars * starSizePx + filledStars * gapPx
    }
}
```

open/final

Класс может наследоваться не только от абстрактных классов, но и от обычных, при условии, что они не являются final.

Класс может быть либо `final`, что означает, что от этого класса нельзя унаследоваться, либо `open`, что, соответственно, позволяет от него наследоваться.

В Kotlin нет ключевого слова `final`, но есть `open`. Каждый класс и метод, не помеченный модификатором `open` является `final` соответственно от него нельзя наследоваться/нельзя его переопределить.

```
open class Animal {
    open fun makeSound() {
        println("Some sound")
    }
}

class Dog : Animal() {
    override fun makeSound() {
        println("Woof!")
    }
}
```

Any

В kotlin все классы наследуются от типа Any.

Any обладает тремя методами:

- equals
- hashCode
- toString

```
The root of the Kotlin class hierarchy. Every Kotlin class has Any as a superclass.

public open class Any {

    Indicates whether some other object is "equal to" this one. Implementations must fulfil the following
    requirements:

    • Reflexive: for any non-null value x, x.equals(x) should return true.
    • Symmetric: for any non-null values x and y, x.equals(y) should return true if and only if y.
      equals(x) returns true.
    • Transitive: for any non-null values x, y, and z, if x.equals(y) returns true and y.equals
      (z) returns true, then x.equals(z) should return true.
    • Consistent: for any non-null values x and y, multiple invocations of x.equals(y) consistently
      return true or consistently return false, provided no information used in equals comparisons on
      the objects is modified.
    • Never equal to null: for any non-null value x, x.equals(null) should return false.

    Read more about equality ↗ in Kotlin.

    public open operator fun equals(other: Any?): Boolean

    Returns a hash code value for the object. The general contract of hashCode is:

    • Whenever it is invoked on the same object more than once, the hashCode method must
      consistently return the same integer, provided no information used in equals comparisons on
      the object is modified.
    • If two objects are equal according to the equals() method, then calling the hashCode method
      on each of the two objects must produce the same integer result.

    public open fun hashCode(): Int

    Returns a string representation of the object.

    public open fun toString(): String
}
```

Any – методы

equals – метод, который сравнивает, равен ли текущий инстанс с другим инстансом, переданным аргументом.

hashCode – метод, который считает hash-функцию от объекта и считает какое-то число. Это удобно использовать при сравнении объектов и в структурах данных, но бывают коллизии, о которых мы еще потом поговорим

toString – метод, который приводит объект к строке, это нужно обычно для вывода текстового представления объекта в логах.

Сравнение

В Kotlin есть оператор ==, который:
- У объектов вызывает метод equals;
- Примитивы сравнивает сразу по значению.

Как мы знаем, каждый объект в Kotlin является наследником Any, а значит каждый класс также в каком-либо виде реализует метод equals.

В случае, если метод не переопределен ни в одном из наследников, объекты будут сравниваться по ссылке (если ссылки ведут на один и тот же инстанс, значит они равны, иначе – нет).

Можно сравнить объекты по ссылке самостоятельно, с помощью оператора ===.

```
class User(
    val name: String,
    val surname: String,
) {
    override fun equals(other: Any?): Boolean {
        if (this === other) return true
        if (javaClass != other?.javaClass) return false

        other as User

        if (name != other.name) return false
        if (surname != other.surname) return false

        return true
    }

    ...
}
```

equals и hashCode

Можно дать следующее определение функции hashCode:

Предположим, что x и y – некоторые объекты.

Если `x == y`, то и `x.hashCode() == y.hashCode()`.

Именно по такому принципу и должна строится реализация функции hashCode(),

никаких больше условий нет.

Выводы:

- Если `x.hashCode() != y.hashCode()`, то `x != y`, что прямо вытекает из определения.
- Если `x.hashCode() == y.hashCode()`, то это ничего не значит

equals и hashCode

hashCode нужен для ускорения.

Посчитать числа гораздо проще,

чем сравнивать все поля класса.

Это актуально для структур данных.

```
class User(
    val name: String,
    val surname: String,
) {
    ...

    override fun hashCode(): Int {
        var result = name.hashCode()
        result = 31 * result + surname.hashCode()
        return result
    }
}
```

data class

data class – класс, который на этапе компиляции генерирует дополнительные методы:
equals, hashCode, toString, copy и component1, ..., componentN.

equals, hashCode, toString у нас были в Any, но в данном случае они генерируют верные реализации, которые будут работать "из коробки" и будут зависеть от полей класса.

```
data class User(
    val name: String,
    val surname: String,
)

val user1 = User( name: "Name 1",  surname: "Surname 1")
val user2 = User( name: "Name 1",  surname: "Surname 1")

assert(user1 == user2) // если убрать модификатор data,
// то проверка не пройдет

// data class генерирует метод copy
val modifiedUser2 = user2.copy(surname = "Modified surname 2")

println(user1.toString) // выведутся поля name и surname
```

Деструктуризация

data class автоматически создаёт методы componentN() для каждого свойства в порядке их объявления. Эти методы нужны для деструктуризации объектов.

```
data class User(val name: String, val age: Int)

val user = User( name: "Alice",  age: 30)
val (n, a) = user  // вызывает component1() и component2()
```

Ограничения data class

- Такой класс обязательно должен содержать поля в конструкторе (иначе ему просто нечего будет генерировать);
- Нельзя унаследоваться от data class (data class при этом может наследоваться). Это связано с copy, мы не сможем реализовать equals/copy из-за наследования.

enum class

enum class — это способ задать перечисление фиксированного набора значений, каждое из которых является объектом. Инстансы enum class-а создаются только в скоупе объявленного enum class.

```
enum class Color {
    Red,
    Green,
    Blue,
}

enum class ColorWithCode(val hexCode: Int) {
    Red( hexCode: 0xff0000),
    Green( hexCode: 0x00ff00),
    Blue( hexCode: 0x0000ff),
}
```

enum class

Удобно использовать для описания заранее заданного ограниченного набора состояний.

```
enum class ButtonStyle {
    Primary,
    Secondary,
}

class Button() : ViewGroup {
    ...

    var style: ButtonStyle

    ...

    private fun resolveBackgroundColor(style: ButtonStyle) = when (style) {
        ButtonStyle.Primary -> context.getColor(R.attr.color_accent)
        ButtonStyle.Secondary -> context.getColor(R.attr.color_secondary)
    }
}
```

sealed class/interface

В kotlin есть абстрактные классы и интерфейсы, которые могут иметь ограниченное количество наследников. Это sealed class/interface (sealed от англ. запечатанный).

Это "расширение" концепции enum class-ов – зафиксированы все возможные наследники класса, но создать их можно в любом месте.

```
sealed class Result

data class Loaded(val data: String) : Result()
data class Error(val e: Throwable) : Result()
object NotModified : Result()

fun printResult(result: Result) {
    when (result) {
        is Loaded -> println(result.data)
        is Error -> Log.e("Tag", result.e)
        is NotModified -> Unit
    }
}
```

object

object — это способ создать единственный экземпляр чего-либо — то есть синглтон.

У object есть только один инстанс класса, до которой мы имеем доступ из любой точки программы в рамках модификатора доступа.

В object поля могут быть помечены как const – это примитивы и строки. Суть const полей в том, что они вообще не будут где-то хранится и просто подставляются в места использования при компиляции программы.

```
object Logger {
    fun log(msg: String) {
        println("Log: $msg")
    }
}
```

companion object

В обычный класс можно добавить companion object, который может содержать статические элементы, как у Java.

```
class ProductCardFragment() : Fragment {
    ...

    companion object {
        const val ID_ARG_KEY = "id"
    }
}

// нам не нужно создавать сущность, чтобы
// получить это значение, оно доступно отовсюду,
// но привязано к пространству класса.
ProductCardFragment.ID_ARG_KEY
```

nested/inner classes

	Вложенный (nested)	Внутренний (inner)
Объявляются внутри другого класса	Да	Да
Модификатор объявления класса	(по умолчанию)	inner
Имеют доступ к private-членам класса	Да (статическим)	Да
Требуют для создания экземпляр внешнего класса	Нет	Да
Имеют ссылку на экземпляр внешнего класса	Нет	Да

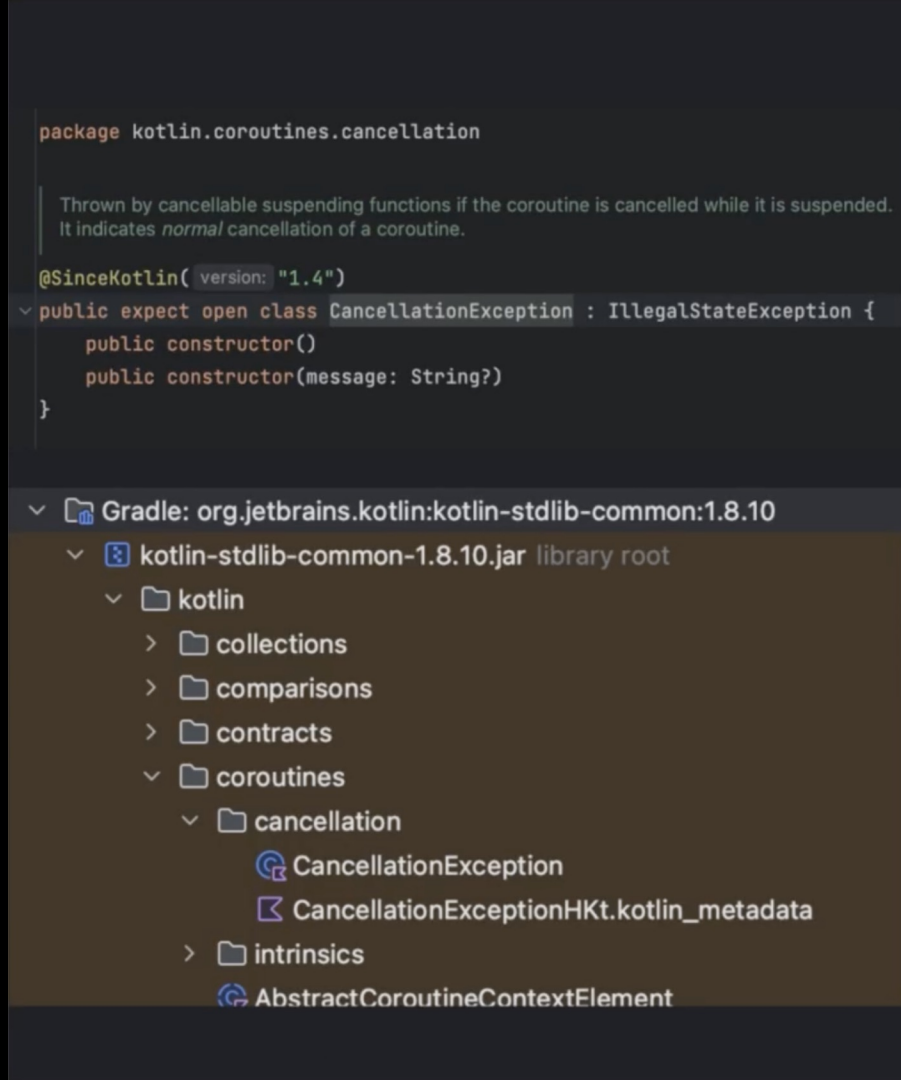
Пакеты

Пакеты (packages) позволяют сгруппировать файлы исходного кода.

Пакет файла может не совпадать с путем к файлу.

Можно определять функции и поля на уровне пакета. (При компиляции обернется в object).

С помощью * в импортах можно импортировать все классы пакета.



Модификаторы доступа

	Члены класса	Объявления на уровне пакета
`public` (по умолчанию)	Доступны везде	Доступны везде
`internal`	Доступны внутри модуля	Доступны внутри модуля
`protected`	Доступны в подклассах	-
`private`	Доступны из того же класса	Доступны из того же файла

extension

Extension-функции позволяют расширять функциональность класса, даже не имея доступа к нему.

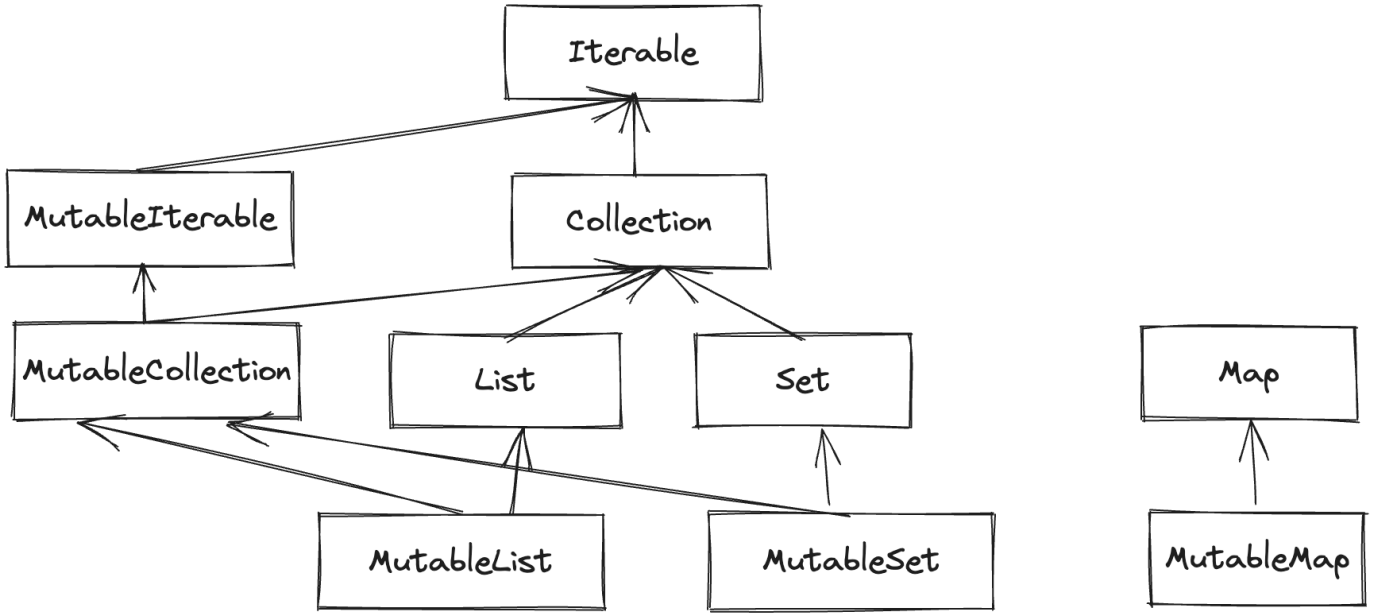
```
data class User(
    val name: String,
    val surname: String,
)

fun User.fullName() = "$name $surname"

// or

val User.fullName
    get() = "$name $surname"
```

Коллекции


Iterable
MutableIterable
Collection
MutableCollection
List
Set
Map
MutableList
MutableSet
MutableMap

list и set

List – список элементов, не ограниченный по размеру.

java.util.List<T>

ArrayList<T> – реализация основана на массиве (arrayListOf, listOf, mutableListOf)
LinkedList<T> – связный список (linkedListOf)

Set — это коллекция уникальных элементов, то есть без повторов.

java.util.Set<T>

HashSet<T> – сет
LinkedHashSet – сет, отслежвающий порядок добавления (setOf, mutableSetOf)

generic

Generic позволяют использовать единую логику для разных типов объектов.

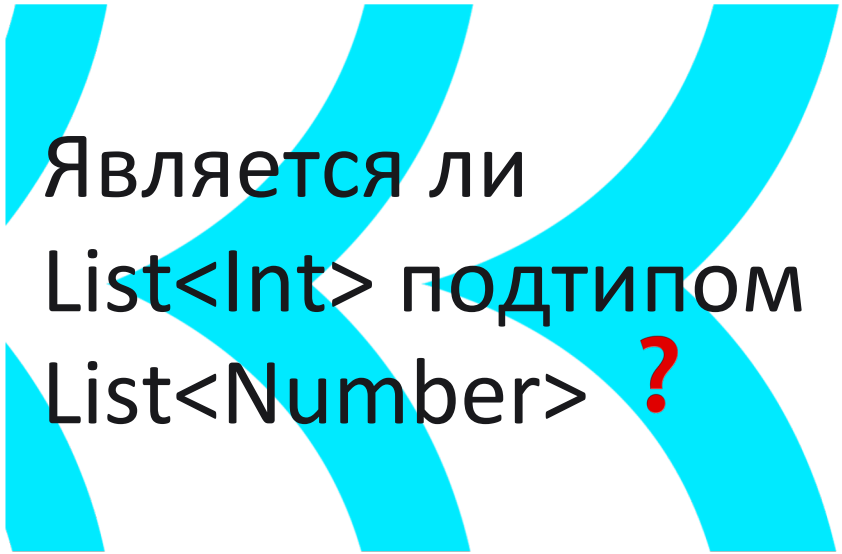
Они могут определяться для всего класса, для конкретной функции или поля (дополнительный механизм абстракции).

Работают по принципу стирания типов (type erasure) – информация о generic-типе на хранится в скомпилированном коде: данные хранятся в виде Any, а в местах использования – приводятся к нужному типу. Исключение – inline-функции с reified-параметром.

```
public interface List<E> extends Collection<E> {
```

```
val list: List<Int> = listOf(1, 2, 3)
```

Generic-типы: вариация

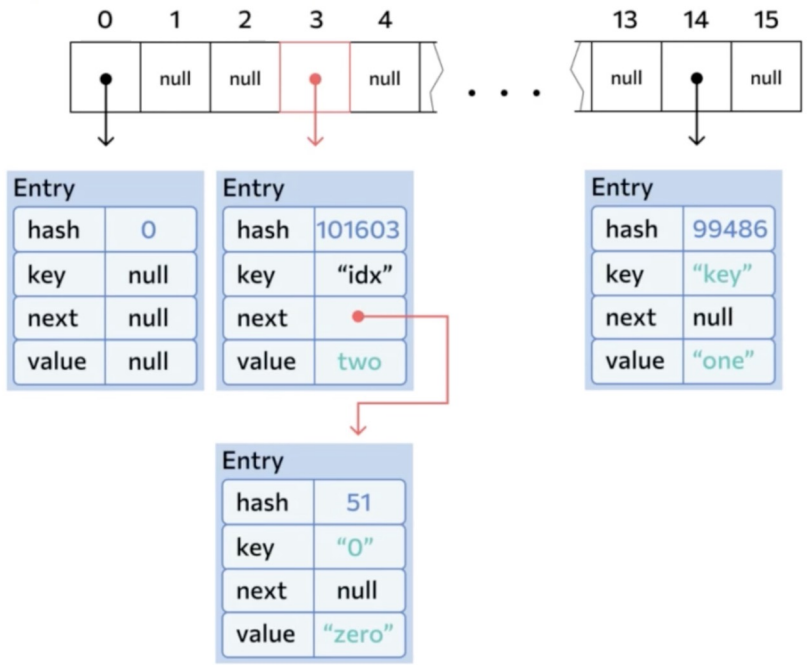


- тип Generic - инвариантный, если
  B подтип A ↛ Generic<B> подтип Generic<A>
- тип Generic - ковариантный, если
  B подтип A ↦ Generic<B> подтип Generic<A>
- тип Generic - контравариантный, если
  B подтип A ↦ Generic<A> подтип Generic<B>
- Generic<T> - инвариантный
  Generic<out T> - ковариантный
  Generic<in T> - контравариантный

HashMap

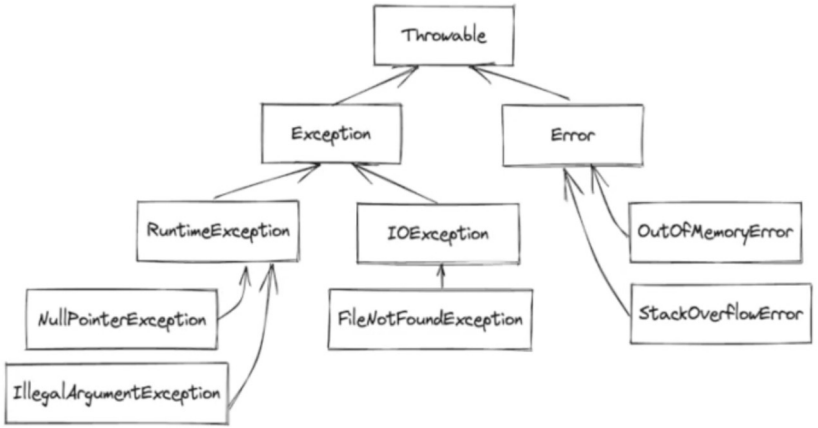
HashMap позволяет *быстро* получать значение по ключу.

hashCode ключа определяет бакет, где хранится значение



Исключения

Исключения – указывают на появление нештатной ситуации. Они прерывают исполнение программы.



Исключения

Исключения можно перехватить, обработать и продолжить выполнение с помощью **try/catch**.

```
try {
    val a = list[5]
    ...
} catch (e: NumberFormatException) {
    // Ошибка формата числа
} catch (e: IndexOutOfBoundsException) {
    // Индекс вне диапазона
}
```

finally позволяет выполнить действие независимо от исключений. Чаще всего используется для очистки ресурсов.

```
val stream = inputFile.inputStream()
try {
    val text = stream.reader().readText()
    println(text)
} catch (e: IOException) {
    println("Read error: ${e.message}")
} finally {
    stream.close()
}
```

Аннотации

Аннотации добавляют Meta-информацию к коду.

Она может быть как просто информативной, так и служить для какой-то задачи на этапе компиляции программы или в runtime.

```
object Utils {
    @JvmStatic
    fun sayHello() = println("Hello")
}

@Deprecated( message: "Используй newFunction()")
fun oldFunction() {
    ...
}

class Repository @Inject constructor() {
    // ...
}
```

Вопросы

Спасибо за внимание!